

Lecture 26: Window Functions II

LAG, LEAD, Running Totals, Moving Averages

Dr. Innocent Nyalala
IIT Madras Zanzibar



D0.jpg

School of Engineering and Science
IIT Madras Zanzibar

6 May 2026

Lecture Outline

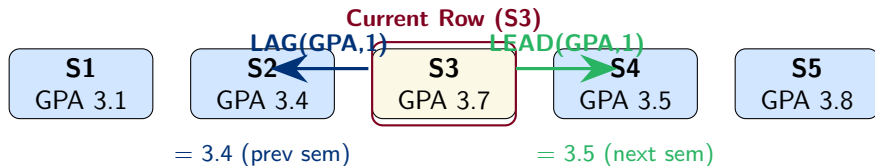
Today's Topics

- 1 LAG and LEAD — look back & forward
- 2 LAG() example — GPA semester change
- 3 LEAD() example — next month revenue
- 4 SUM OVER — running (cumulative) total
- 5 SUM OVER PARTITION BY — per-user total
- 6 Moving average — ROWS BETWEEN frame
- 7 Combining multiple window functions

Schemas Used

```
SalesData(  
    SaleID, Month, Revenue)  
  
StudentGrades(  
    StudentID, Semester, GPA)
```

LAG and LEAD — Looking Back and Forward



- ▶ **LAG(col, n)** — value n rows **before**
- ▶ Default offset = 1 row back

- ▶ **LEAD(col, n)** — value n rows **ahead**
- ▶ NULL if no prior/next row exists

LAG() — GPA Change Semester over Semester

```
SELECT
  StudentID, Semester, GPA,
  LAG(GPA) OVER (
    PARTITION BY StudentID
    ORDER BY Semester
  ) AS PrevGPA,
  GPA - LAG(GPA) OVER (
    PARTITION BY StudentID
    ORDER BY Semester
  ) AS GPACheck
FROM StudentGrades;
```

ID	Sem	GPA	Prev	Δ
S01	S1	3.1	NULL	—
S01	S2	3.4	3.1	+0.3
S01	S3	3.7	3.4	+0.3
S01	S4	3.5	3.7	-0.2

Immediate trend per student!

LEAD() — Preview Next Month's Revenue

```
SELECT
  Month, Revenue,
  LEAD(Revenue, 1, 0) OVER (
    ORDER BY Month
  ) AS NextMonthRev,
  LEAD(Revenue, 1, 0) OVER (
    ORDER BY Month
  ) - Revenue AS RevDelta
FROM SalesData;
```

Month	Rev	Next	Δ
Jan	120K	145K	+25K
Feb	145K	98K	-47K
Mar	98K	160K	+62K
Apr	160K	0	-

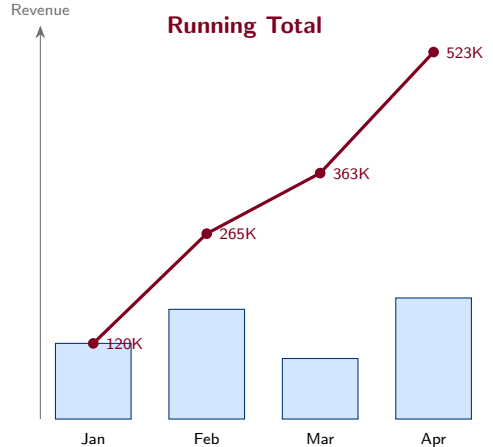
Useful for forecasting dashboards!

Third argument 0 = **default** if no next row

SUM OVER — Cumulative Running Total

```
SELECT
  Month, Revenue,
  SUM(Revenue) OVER (
    ORDER BY Month
    ROWS BETWEEN
      UNBOUNDED PRECEDING
      AND CURRENT ROW
  ) AS RunningTotal
FROM SalesData;
```

- ▶ Accumulates **row by row**
- ▶ Month 3 total = $M1 + M2 + M3$



SUM OVER PARTITION BY — Per-Student Running Total

```
SELECT StudentID, Semester, Amount,  
       SUM(Amount) OVER (  
         PARTITION BY StudentID  
         ORDER BY Semester  
       ) AS CumulativeSpend  
FROM StudentOrders;
```

Partition 1

Alice:

S1: 200	S2: 350	S3: 150
cum: 200	550	700

Partition 2

Bob:

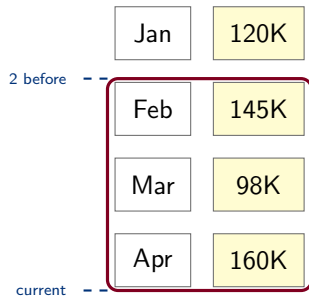
S1: 180	S2: 220	S3: 300
cum: 180	400	700

Each student's running total resets independently

Moving Average — ROWS BETWEEN Sliding Window

```
SELECT Month, Revenue,  
       AVG(Revenue) OVER (  
         ORDER BY Month  
         ROWS BETWEEN  
           2 PRECEDING  
           AND CURRENT ROW  
       ) AS MovAvg3  
FROM SalesData;  
  
-- Smooths out spikes over 3  
   months
```

Includes current row + 2 rows before



Window for Apr

$$\text{Avg} = (145 + 98 + 160) / 3 = 134.3\text{K}$$

Combining Window Functions

```
SELECT
  StudentID, Semester, GPA,
  ROW_NUMBER() OVER (PARTITION BY StudentID ORDER BY Semester) AS
    SemNum,
  LAG(GPA)      OVER (PARTITION BY StudentID ORDER BY Semester) AS
    PrevGPA,
  GPA - LAG(GPA) OVER (PARTITION BY StudentID ORDER BY Semester) AS
    Change,
  SUM(GPA)      OVER (PARTITION BY StudentID ORDER BY Semester) AS
    CumGPA
FROM StudentGrades;
```

StudentID	Sem	GPA	SemNum	PrevGPA	Change	CumGPA
S01	S1	3.1	1	NULL	–	3.1
S01	S2	3.4	2	3.1	+0.3	6.5
S01	S3	3.7	3	3.4	+0.3	10.2

Think-Pair-Share (5 min)

Two Problems

Problem 1: Using StudentGrades(StudentID, Semester, GPA):

Write a query showing each student's GPA, their **previous semester GPA**, and whether the GPA went **up, down, or stayed the same**.

Problem 2: Using SalesData(Month, Revenue):

Compute a **3-month moving average** and also the **cumulative total** in one query.

Hint for P1

```
CASE
  WHEN GPA > LAG(GPA)... THEN 'UP'
  WHEN GPA < LAG(GPA)... THEN 'DOWN'
  ELSE 'SAME'
END AS Trend
```

Hint for P2

```
AVG(Revenue) OVER (
  ORDER BY Month
  ROWS BETWEEN 2 PRECEDING
  AND CURRENT ROW
) AS MovAvg,
SUM(Revenue) OVER (
  ORDER BY Month
) AS RunTotal
```

Summary

Key Takeaways

- ▶ **LAG/LEAD** — access adjacent rows
- ▶ **SUM OVER** — running cumulative total
- ▶ **AVG OVER + ROWS BETWEEN** — moving average
- ▶ Multiple window functions **in one query**
- ▶ **PARTITION BY** keeps **groups separate**

AVG + ROWS — moving avg

SUM OVER — cumulative

LEAD(col) — look forward

LAG(col) — look back

Next: L27 — Stored Procedures and Triggers